ГУАП

КАФЕДРА № 41

ОТЧЕТ
ЗАЩИЩЕН С ОЦЕНКОЙ

ПРЕПОДАВАТЕЛЬ

Ассистент		А.С. Раскопина
должность, уч. степень, звание	подпись, дата	инициалы, фамилия

ОТЧЕТ О ЛАБОРАТОРНОЙ РАБОТЕ №5

Создание запросов на сервере MySQL

по курсу: Базы данных

РАБОТУ ВЫПОЛНИЛ

СТУДЕНТ ГР. № 4417		М. П. Курбатов
	подпись, дата	инициалы, фамилия

Санкт-Петербург 2026

Лабораторная работа № 7. Программная реализация пользовательских функций

Цель работы

Создать две хранимые функции для БД «ГИБДД»: функцию с ветвлением и функцию с циклом. Использовать параметры, локальные и пользовательские сессионные переменные; выполнить положительные и отрицательные тесты.

1 Вариант и база данных

Вариант 13 — «ГИБДД». Инспекция дорожного движения Хоббитонии регистрирует автомобили граждан и учитывает ДТП. У автомобиля хранятся марка, мощность, максимальная скорость, год выпуска, стоимость и текущий владелец. Для ДТП фиксируются дата, время, адрес и участвующие автомобили. Один гражданин может владеть несколькими автомобилями и продавать их другим гражданам.

Используется существующая база gibdd из работ № 4–5. Контрольный набор: 10 граждан, 10 автомобилей, 5 ДТП, 10 участий и 5 продаж. Текст ниже содержит ожидаемые значения для этого набора. Свои результаты подтверждаются снимками Workbench в отведённых местах. SQL предварительно проверен на MySQL 8.0.46.

На рисунке 1 необходимо показать пять таблиц, все поля и шесть внешних ключей. Полные названия должны читаться, линии связей не должны пересекаться. Таблица 1 перечисляет связи базы.

Вставить свой снимок: ваша EER Diagram с пятью таблицами и всеми связями



Рисунок 1 — Схема базы gibdd в MySQL Workbench

Таблица 1 — Связи базы данных

Внешний ключ	Родительский ключ
cars.ownerId	citizens.citizenId
accident_cars.accidentId	accidents.accidentId
accident_cars.carId	cars.carId
sales.carId	cars.carId
sales.sellerId	citizens.citizenId
sales.buyerId	citizens.citizenId

В accident_cars составной первичный ключ (accidentId, carId). Он не допускает повторения одного автомобиля в одном ДТП. В этой таблице считаются участия, а в accidents — сами происшествия. Число участий не определяет виновность водителя.

2 Функция числа ДТП автомобиля

lr7_simple_car_count(p_car_id) нужна инспектору, который проверяет число зарегистрированных участий автомобиля в ДТП. Параметр p_car_id INT — внутренний код автомобиля. Возвращается одно целое число: 0 или больше — корректный результат; -1 — неверный или отсутствующий код. Значение -1 специально зарезервировано для ошибки, потому что количество ДТП не бывает отрицательным. Функция не определяет виновность.

Алгоритм рисунка 2: проверить NULL и знак кода; проверить существование автомобиля; при любой ошибке вернуть -1; иначе посчитать строки accident_cars для данного carId и вернуть число. Существующий автомобиль без ДТП даёт 0. Локальная v_exists используется для проверки существования, v_count — для результата. Это отличает отсутствие автомобиля от отсутствия происшествий.

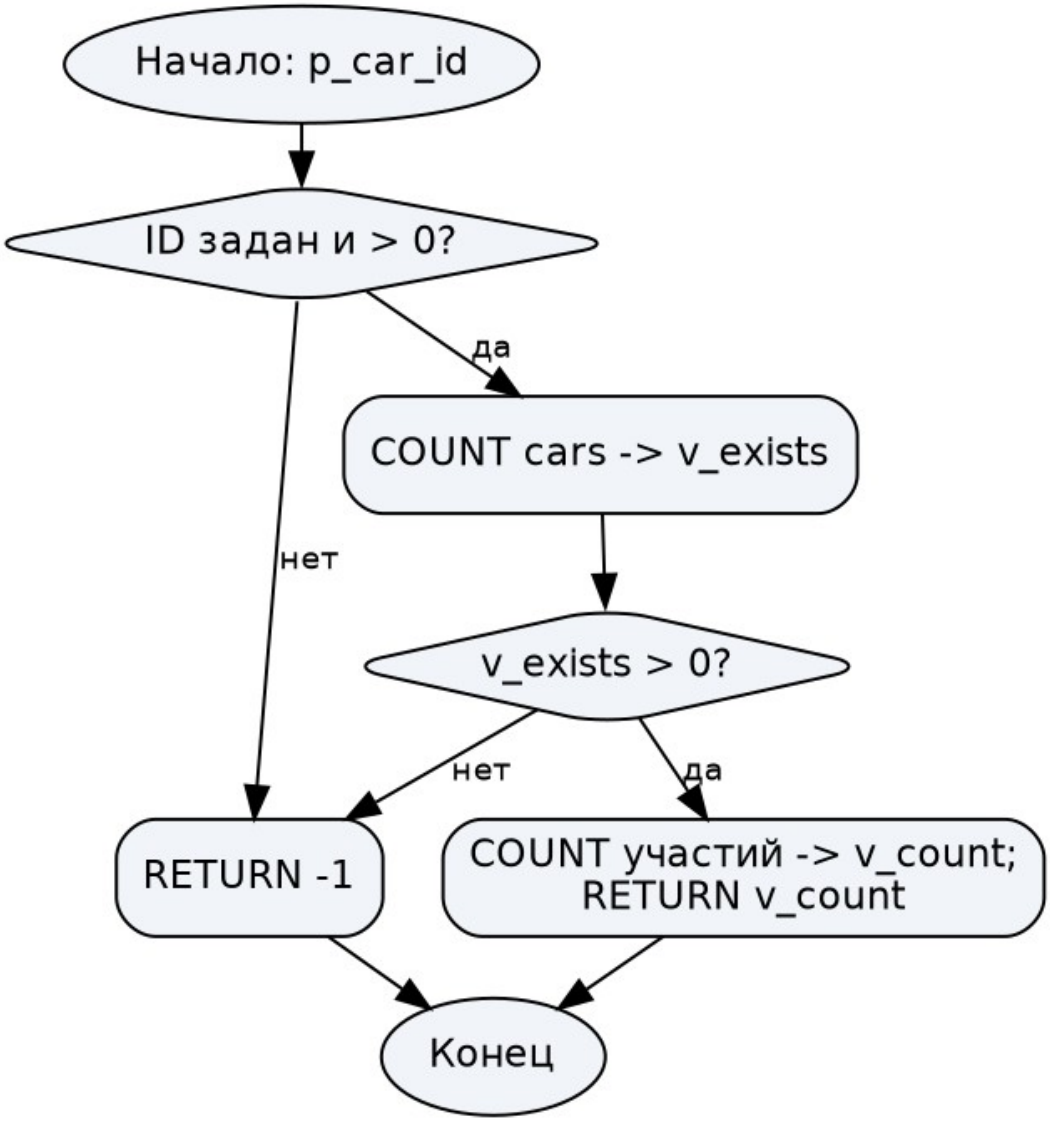


Рисунок 2 — Алгоритм функции lr7_simple_car_count

Листинг 1 — Функция с ветвлением

```
USE gibdd;
DELIMITER $$
CREATE FUNCTION lr7_simple_car_count(p_car_id INT)
```

```
RETURNS INT
NOT DETERMINISTIC
READS SQL DATA
BEGIN
    DECLARE v_exists INT DEFAULT 0;
    DECLARE v_count INT DEFAULT 0;
    -- 1. Отрицательный результат -1 означает ошибочный параметр.
    IF p_car_id IS NULL OR p_car_id <= 0 THEN
        RETURN -1;
    END IF;
    -- 2. Положительный код тоже нужно проверить на существование.
    SELECT COUNT(*) INTO v_exists
    FROM gibdd.cars WHERE carId = p_car_id;
    IF v_exists = 0 THEN
        RETURN -1;
    END IF;
    -- 3. Считаем участия; составной ключ исключает дубли одного ДТП.
    SELECT COUNT(*) INTO v_count
    FROM gibdd.accident_cars WHERE carId = p_car_id;
    RETURN v_count;
END$$
DELIMITER ;
```

3 Функция годового числа ДТП с циклом

lr7_simple_year_count(p_year) нужна аналитику ГИБДД для получения годового итога внутри SELECT. Параметр — год от 1901 до 2155 включительно. Возвращается целое число ДТП либо -1 при неверном годе. Диапазон — принятое учебное правило, а не предел типа DATE. Алгоритм рисунка 3 повторяет знакомый подсчёт по 12 месяцам из процедуры работы № 6, но вместо вывода строки использует RETURN.

Локальные v_month, v_count и v_total имеют те же роли: месяц, число ДТП за месяц и сумма. Перед вызовами используются @report_year и @car_id — пользовательские переменные текущего соединения. Они доступны вне тела функции и передаются в параметры; локальные переменные снаружи недоступны. Это не настройки @@GLOBAL сервера. Функции не изменяют эти переменные и не меняют таблицы.

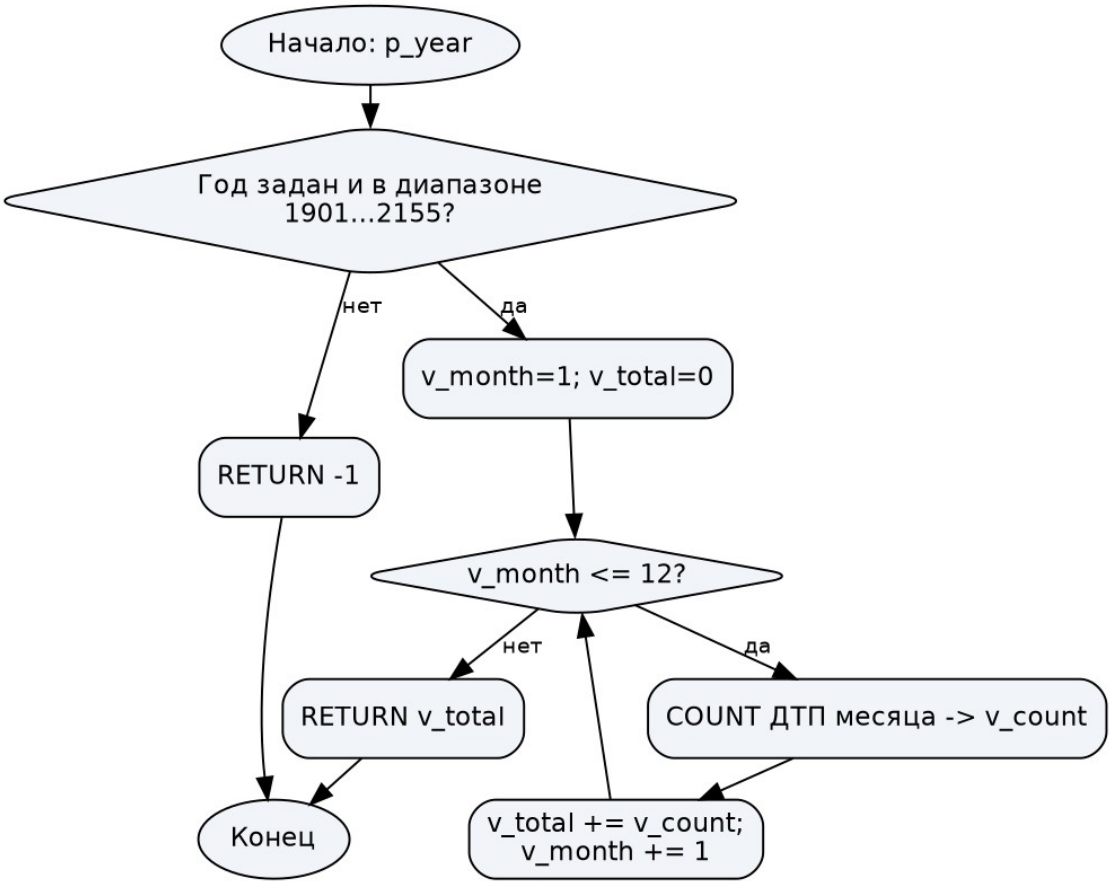


Рисунок 3 — Алгоритм функции lr7_simple_year_count

Листинг 2 — Функция с циклом

```
USE gibdd;
DELIMITER $$
CREATE FUNCTION lr7_simple_year_count(p_year INT)
RETURNS INT
NOT DETERMINISTIC
READS SQL DATA
BEGIN
    DECLARE v_month INT DEFAULT 1;
    DECLARE v_count INT DEFAULT 0;
    DECLARE v_total INT DEFAULT 0;
    -- 1. Проверяем год; -1 нельзя спутать с нулём ДТП.
    IF p_year IS NULL OR p_year < 1901 OR p_year > 2155 THEN
        RETURN -1;
    END IF;
    -- 2. Цикл считает происшествия для каждого из 12 месяцев.
    WHILE v_month <= 12 DO
        SELECT COUNT(*) INTO v_count
        FROM gibdd.accidents
        WHERE YEAR(accidentDate) = p_year
          AND MONTH(accidentDate) = v_month;
        SET v_total = v_total + v_count;
        SET v_month = v_month + 1;
    END WHILE;
    -- 3. Функция возвращает одно целое число.
    RETURN v_total;
END$$
DELIMITER ;
```

RETURNS INT объявляет тип результата, RETURN завершает функцию и возвращает число. READS SQL DATA указывает чтение данных. NOT DETERMINISTIC означает, что одинаковый параметр не гарантирует одинаковый

результат после изменения таблиц. Объявлять эти функции DETERMINISTIC было бы неверно. В каждом листинге три поясняющих комментария. Полный файл 02_lab7_functions.sql также содержит безопасную замену только двух одноимённых функций.

4 Список функций и проверка

Список всех функций базы должен быть показан на рисунке 4. Затем рисунки 5–9 подтверждают все случаи из таблицы 2. Значения -1 в отрицательных тестах — предусмотренная диагностика; красной SQL-ошибки при этом не возникает.

Листинг 3 — Список функций

```
SELECT ROUTINE_NAME, ROUTINE_TYPE
FROM information_schema.ROUTINES
WHERE ROUTINE_SCHEMA = 'gibdd'
  AND ROUTINE_TYPE = 'FUNCTION'
ORDER BY ROUTINE_NAME;
```

Вставить свой снимок: запрос списка и обе функции lr7_simple_car_count / lr7_simple_year_count

МЕСТО ДЛЯ ВАШЕГО СКРИНШОТА № 4

Рисунок 4 — Список функций базы gibdd

Таблица 2 — Ожидаемые результаты функций

Функция	**Вход**	**Результат**
car_count	1 / 9	2 / 0
car_count	NULL / 0 / -1 / 999999	по -1
year_count	2025 / 2024	5 / 0
year_count	1901 / 2155	0 / 0
year_count	NULL / 1900 / 2156	по -1

Проверки кода автомобиля показывают обе ветви проверок и допустимый нулевой результат. Годы 1901 и 2155 принимаются, 1900 и 2156 отклоняются; NULL проверяется отдельно. Запросы приведены в листинге 4. Для кода 999999 предполагается, что такой автомобиль отсутствует; при другом наборе данных нужен действительно отсутствующий положительный код.

Листинг 4 — Вызовы функций

```
USE gibdd;
-- Рисунок 4: все функции базы.
SELECT ROUTINE_NAME, ROUTINE_TYPE FROM information_schema.ROUTINES
WHERE ROUTINE_SCHEMA = 'gibdd' AND ROUTINE_TYPE = 'FUNCTION'
ORDER BY ROUTINE_NAME;
-- Рисунок 5: число ДТП автомобиля: ожидаются 2 и 0.
```

```
SET @car_id = 1;
SELECT @car_id AS carId, lr7_simple_car_count(@car_id) AS accident_count;
SET @car_id = 9;
SELECT @car_id AS carId, lr7_simple_car_count(@car_id) AS accident_count;
-- Рисунок 6: ошибки кода: ожидаются четыре значения -1.
SELECT lr7_simple_car_count(NULL) AS null_id,
       lr7_simple_car_count(0) AS zero_id,
       lr7_simple_car_count(-1) AS negative_id,
       lr7_simple_car_count(999999) AS missing_id;
-- Рисунок 7: итоги за годы: ожидаются 5 и 0.
SET @report_year = 2025;
SELECT @report_year AS report_year, lr7_simple_year_count(@report_year) AS
accident_count;
SET @report_year = 2024;
SELECT @report_year AS report_year, lr7_simple_year_count(@report_year) AS
accident_count;
-- Рисунок 8: допустимые границы: ожидаются 0 и 0.
SELECT lr7_simple_year_count(1901) AS min_year,
       lr7_simple_year_count(2155) AS max_year;
-- Рисунок 9: ошибки года: ожидаются три значения -1.
SELECT lr7_simple_year_count(NULL) AS null_year,
       lr7_simple_year_count(1900) AS below_min,
       lr7_simple_year_count(2156) AS above_max;
```

Вставить свой снимок: SET @car_id, затем SELECT функции: результаты 2 и 0

МЕСТО ДЛЯ ВАШЕГО СКРИНШОТА № 5

Рисунок 5 — Число ДТП автомобилей № 1 и № 9

Вставить свой снимок: один SELECT с четырьмя результатами -1

МЕСТО ДЛЯ ВАШЕГО СКРИНШОТА № 6

Рисунок 6 — Неверные и отсутствующий коды автомобиля

Вставить свой снимок: SET @report_year, затем SELECT функции: результаты 5 и 0

МЕСТО ДЛЯ ВАШЕГО СКРИНШОТА № 7

Рисунок 7 — Годовая функция для 2025 и 2024 годов

Вставить свой снимок: один SELECT: для 1901 и 2155 годов по 0

МЕСТО ДЛЯ ВАШЕГО СКРИНШОТА № 8

Рисунок 8 — Допустимые границы года в функции

Вставить свой снимок: один SELECT: NULL, 1900 и 2156 дают -1

МЕСТО ДЛЯ ВАШЕГО СКРИНШОТА № 9

Рисунок 9 — Неверные годы в функции

На проверочном сервере успешно выполнены шесть тестов первой функции и семь второй. Результаты совпали с таблицей 2, исходные таблицы сохранены. Эти протоколы приложены отдельно; свои скриншоты Workbench необходимо вставить в рисунки 4–9.

5 Вывод

Пользовательские функции позволяют использовать показатели предметной области прямо в SELECT. Инспектор получает число участий автомобиля, а аналитик — годовое число ДТП. Эти показатели имеют разный смысл: первая функция читает связи автомобилей с происшествиями, вторая — сами происшествия. Такое разделение помогает избежать двойного подсчёта при анализе данных ГИБДД.

Проверка существования автомобиля отличает ошибочный код от автомобиля без ДТП. Для ошибочного кода возвращается -1, а отсутствие происшествий даёт 0. Этот договор о результате необходимо учитывать в программе, которая вызывает функцию. Он делает учебную реализацию короткой, но требует проверки отрицательного значения перед использованием числа как статистического показателя.

Функция с циклом показывает последовательный расчёт: для каждого месяца получается число ДТП, затем оно добавляется к общей сумме. Подход намеренно повторяет простой алгоритм из шестой работы, поэтому различие подпрограмм сводится к способу вызова и выдачи результата. Процедура вызывается CALL и выводит строку; функция участвует в SELECT и возвращает одно значение через RETURN. Граничные и отрицательные тесты подтверждают работу проверок. Обе

функции только читают данные.

Список использованных источников

1. Аграновский А. В., Боженко В. В., Турнецкая Е. Л. Разработка и администрирование базы данных с открытым исходным кодом. ГУАП, 2022. Подраздел 7.6, с. 133–134; приложение В, вариант 13.

2. MySQL 8.0 Reference Manual: CREATE PROCEDURE, CREATE FUNCTION, IF, WHILE, RETURN, Local Variables, User-Defined Variables. URL: https://dev.mysql.com/doc/refman/8.0/en/.

3. Курбатов М. П. Отчёт о лабораторной работе № 5 «Создание запросов на сервере MySQL». Предоставленный исходный отчёт, 2026.

4. ГОСТ 7.32–2017. Отчёт о научно-исследовательской работе. ГОСТ Р 2.105 –2019. Общие требования к текстовым документам.